IITR-AS-2601

IITR-AS-2601 — Final Capstone

GRADE REPORT

STUDENT

Yuvraj Singh Pundir

6757527 · yuvrajpundir2005@gmail.com

TOTAL MARKS	SCALED MARKS
87	**8.7**
out of 100	out of 10

SECTION SCORES

Part 1 - Dataset Design & RAG Core		**25 / 30** 83%
Part 2 - LangGraph Agent with Tools, Memory & Guardrails		**25 / 30** 83%
Part 3 - Evaluation, Observability & FastAPI Deployment		**20 / 20** 100%
Part 4 - Resilience & Interoperability		**17 / 20** 85%

SECTION FEEDBACK

part1 — Part 1 - Dataset Design & RAG Core — 25 / 30

This submission demonstrates substantial engineering effort well beyond the baseline Part 1 requirements — including a full LangGraph agent, FastAPI backend, MCP server, guardrails, and multiple evaluation harnesses (RAG-triad, mock-vs-real LLM judges, demonstration scripts). The core Part 1 deliverables are present and mostly functional: a seeded dataset generator with correct category/status coverage, a knowledge base spanning all 12 required topics across 38 authored documents, and two properly separated chunking strategies (fixed + sentence-based) embedded into distinct Chroma collections with SentenceTransformers. The main weaknesses are internal inconsistencies between analysis and implementation: the dataset generator's delayed_shipment percentage validation is acknowledged as unenforced in code comments rather than actually implemented as a regenerate-on-failure loop, and the empirically calibrated threshold (0.3282) derived in the write-up is never applied in the actual evaluation script, which still hardcodes 0.1 — a threshold the student's own analysis proves is wrong. The Precision@3/Recall@3 comparison is present with genuine per-query arithmetic for both strategies, but the results are identical across strategies, likely due to minimal expected_sources granularity, weakening the comparative value. Going forward, ensure the calibrated values from analysis are wired back into the production code, and add a proper deterministic seed-search loop for the delayed_shipment percentage rather than relying on comments and hope.

part2 — Part 2 - LangGraph Agent with Tools, Memory & Guardrails — 25 / 30

This is a substantial, professionally engineered submission with strong software engineering practices (retry policies, timeouts, SQLite checkpointing, structured logging, FastAPI wrapping, MCP interoperability). The LangGraph graph clearly exceeds the node/conditional-edge requirements and both routing branches are demonstrated. Persisted memory is well-evidenced with an explicit cross-instantiation test proving disk persistence and correct recovery of omitted context. PII masking and prompt-injection guardrails are convincingly demonstrated firing with before/after evidence at multiple pipeline boundaries. The weaker areas are the escalation-score threshold justification (present formula but no stated/justified threshold tied to the dataset's actual distribution) and the output-side groundedness/refusal guardrail, which is not clearly demonstrated firing inside the actual agent graph response path (the similarity-threshold fallback exists but lives in the separate Task 4/5 RAG evaluation script rather than being clearly wired as an output guardrail in generate_response). A fresh-conversation "state absent" transcript, separate from the multi-turn demo, is also not clearly provided. Tightening these two specific proof points would meaningfully raise the score.

part3 — Part 3 - Evaluation, Observability & FastAPI Deployment — 20 / 20

This is a strong, well-engineered submission for Part 3. The FastAPI layer is comprehensive (well beyond the required 2 endpoints) with clean Pydantic models, resumable LangGraph checkpoints, and a working frontend to exercise it. The structured logging module is a highlight: it's a proper contextmanager producing one JSONL entry per request with trace_id/timing, and it explicitly reuses the PII masking guardrail on both request and response text with a safe failure mode, verified by an actual demonstration script that inspects the on-disk log file for leaked values. The RAG-triad evaluation fully meets the 15-query, 12-topic, MOCK_LLM-judge requirement with per-query and aggregate scores in both JSON/CSV, and the student went further by adding a real-LLM comparison and a thoughtful analysis writeup. Minor deductions/notes: the main.py file provided is truncated (upload endpoint body not fully visible) and requirements.txt is corrupted (UTF-16 artifact), though this doesn't affect the graded criteria. Overall this is excellent, thorough work with clear engineering discipline and honest documentation of design tradeoffs.

part4 — Part 4 - Resilience & Interoperability — 17 / 20

Strong resilience/interoperability implementation overall: MCP tool exposure, SQLite checkpointing, retry policies, and timeouts are all implemented in real, functioning code (not stubs) and backed by demonstration scripts and JSON evidence files rather than just claims. The MCP client-server round trip works and is genuinely a separate client process, but it uses the deprecated SSE transport at `/sse` instead of the rubric-mandated `/mcp` HTTP transport path, a notable deviation from the explicit task instructions. Checkpointing and resume logic are well-engineered (get_thread_status, resume=True with input=None, frontend resume banner) but the submission doesn't clearly show a node-execution-count/log proof that completed nodes were skipped on resume. Retry and per-node timeout are cleanly demonstrated with realistic failure injection and timing evidence, but a dedicated demonstration of the *global* timeout independently cancelling an overrunning full run (as opposed to a single node) is not clearly present in the shared artifacts. Overall a solid, well-documented Part 4 with minor spec-compliance and evidentiary gaps.